AWS AgentCore Gateway and Policy

Independent public research | Grounded DI LLC | October 2026

An evidence-based review of the boundary between an agent request, a policy decision and a tool effect. AWS documentation, Cedar source analysis and separate synthetic DI² controls are kept distinct.

The reviewed record does not establish a hosted AWS failure, customer incident, security bypass, or product-wide assessment. No AWS account, hosted Gateway, Cedar runtime or external effect was tested.

01 / Evidence map

F1: MCP coverage — DOCUMENTED AWS SCOPE. Tool calls are evaluated by policy; prompt and resource methods are outside that evaluation as documented. Not a demonstrated bypass. [S1]

F2: mode and administrative authority — DOCUMENTED CONFIGURATION. Engine and policy logging modes have different effects; authority to modify Gateway configuration affects the effective boundary. [S2–S3]

F3: Cedar errors — SOURCE ANALYSIS. An erroring policy is skipped by the pinned Cedar authorizer; a potential interaction with a separate permit remains an unexecuted hypothesis. [S7–S8]

F4: decision and effect — UNTESTED IN HOSTED AWS. Documented telemetry is not itself proof of a downstream effect. [S5]

02 / Sources and custody

AWS documentation was retrieved on October 28, 2026 as line-numbered text returned by a retrieval service. Nine captures contain all lines exposed by that service; they are not original HTML, complete media archives, or independent proof of a live page at the retrieval time. The prior record preserves earlier excerpts. [S1–S6]

The AgentCore starter toolkit policy overview was inspected at commit 316fa52303d0652035aaebe3c603a486f0d988bc. Cedar source was inspected at commit 6e4fa639c1a77ab2e1fa822a9bd3b6f0a4228e40. The toolkit is not the managed Gateway implementation. [S7–S9]

The private record contains the harness, local plan and receipts. This public edition is an explanation of that evidence, not a standalone executable replay package.

03 / Tool calls, discovery and resources

AWS documents policy evaluation for MCP tool invocations. The same documentation says prompt and resource methods are allowed regardless of policy mode. This is a defined surface of policy evaluation; it does not prove access without authentication or that other controls are absent. [S1]

Tool listing is a meta action using partial authorization: a tool may appear where some invocation could be allowed, without promising authorization for every set of arguments. [S1]

Counterargument: target authorization, identity controls and network restrictions can operate outside the policy engine. To settle actual exposure, measure each route in an authorized isolated deployment and record each additional control and target response.

04 / Enforcement modes and authority

AWS describes engine ENFORCE as applying authorization decisions and engine LOG_ONLY as evaluating and logging without enforcing them. An individual policy can also be LOG_ONLY in an enforcing engine while other active policies remain effective. [S2–S3]

AWS states that UpdateGateway permission can change the engine mode or remove the association. This is a documented administrative capability, not evidence that an unauthorized user changed the setting. [S2]

Decisive test: capture effective IAM permissions, Gateway configuration, policy state and change events; compare the same synthetic request under each authorized mode. Record decision telemetry and independently observed target invocation.

05 / Identity, validation and errors

AWS documents OAuth-user principals derived from JWT subject claims and IAM principals based on caller identity. Cedar policies can address principal, action, Gateway resource and request context. AWS describes default denial and forbid precedence. [S4, S6]

Schema validation checks referenced actions and types; AWS also describes semantic validation modes and mismatch diagnostics. These are material counterevidence to a broad assertion that malformed inputs automatically become unsafe. [S5–S6]

The record does not execute real Gateway mapping or resolve every missing-attribute, malformed-input, policy-error or policy-update outcome. A hosted test would require the exact policy, schema, identity, input, diagnostic and target effect.

06 / Cedar source: erroring policies

The pinned Cedar authorizer records evaluation errors and, in its default skip mode, treats an erroring policy as not satisfied. Its partial-response decision logic uses satisfied permits and forbids. [S7–S8]

A narrow hypothesis follows: where a forbid errors but a separate permit is satisfied, the failed forbid may not deny and the diagnostic needs attention. This is source analysis, not an executed Cedar result. It is not evidence of a hosted AgentCore bypass. [S7–S8]

The eight proposed Cedar cases were not run: the environment lacked a discoverable Cedar executable and Rust/Cargo toolchain. Exact executable fixtures remained to be finalized and frozen. A real Cedar test would not by itself prove managed Gateway behavior.

07 / What ran locally

Five separately planned DI² Gate checks used a synthetic local target and trusted host-supplied approvals. They recorded: G01 approved invocation once; G02 changed context denied; G03 original prepared argument snapshot retained; G04 repeated dispatch denied; G05 expiry denied before invocation. Each local assertion passed. [E1]

The separate native baseline recorded 99 implementation passes and six reproduced-bug fixtures in 105 core cases, plus 135 unit passes. The six reproduced defects are not six passing safety properties. [E1]

These checks do not execute AWS Gateway or Cedar. The local Gate does not authenticate a real person or bind every external dependency, and a local return receipt does not independently attest an external effect.

08 / Decision, effect and next test

A complete evidence chain would identify the governing policy version and mode, authenticated principal, Gateway and tool identity, exact request and relevant context; then preserve the authorization decision, diagnostics, downstream invocation and an independently observed effect. [S4–S5]

AWS describes metrics and additional trace fields when tracing is enabled. Such records are useful decision evidence, but a decision alone is not a downstream-effect receipt. [S5]

An authorized test should cover ordinary allow and deny, missing attributes, competing permit and forbid, policy changes, mode transitions, tool discovery and non-tool operations. It should use a synthetic target, preserve raw observations, and reconcile skips and errors. No such hosted test was performed.

09 / Source ledger and release verification

S1 Gateway operations: https://docs.aws.amazon.com/bedrock-agentcore/latest/devguide/use-gateway-with-policy.html

S2 Gateway modes: https://docs.aws.amazon.com/bedrock-agentcore/latest/devguide/policy-enforcement-modes.html

S3 Policy-level modes: https://docs.aws.amazon.com/bedrock-agentcore/latest/devguide/policy-test-a-policy.html

S4 Principal and policy concepts: https://docs.aws.amazon.com/bedrock-agentcore/latest/devguide/policy-core-concepts.html

S5 Telemetry: https://docs.aws.amazon.com/bedrock-agentcore/latest/devguide/observability-policy-metrics.html

S6 Validation: https://docs.aws.amazon.com/bedrock-agentcore/latest/devguide/policy-validation-overview.html

S7 Cedar authorizer and S8 partial response: https://github.com/cedar-policy/cedar/tree/6e4fa639c1a77ab2e1fa822a9bd3b6f0a4228e40/cedar-policy-core/src

S9 Toolkit overview: https://github.com/aws/bedrock-agentcore-starter-toolkit/tree/316fa52303d0652035aaebe3c603a486f0d988bc

E1: Grounded DI private v2 record and local test receipts. Historical results reported; not rerun for this edition.